

Chapter 3

CSS Part 3

Box Model, Floating and Positioning

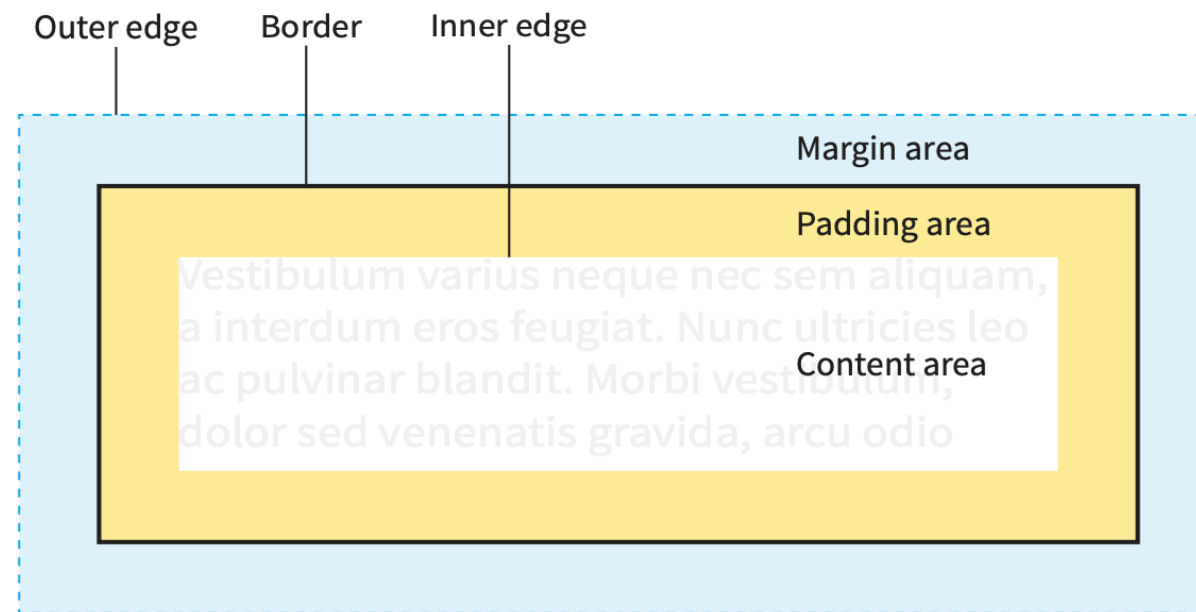
INT150 Web Technologies (2/2025)

Sanit Sirisawatvatana

CSS Box Model

- All HTML elements can be considered as **boxes**.
- The **box model** is used to create the design and layout of web pages.
- CSS **box model** is a **container** which contains multiple properties including borders, margin, padding and the content itself.

Box Model



- **Content area** The content of the box, where text and images appear.
- **Inner edges** The edges of the content area. **The edge of the content area is invisible.**
- **Padding** Clears an area around the content. **The padding is transparent.**
- **Border** The border is a line (or stylized line) that goes around the padding and content.
- **Margin** Clears an area outside the border. **The margin is transparent**, allowing the background of the parent element to show though.
- **Outer edge** The outside edges of the margin area make up the outer edges of the element box.
This is total area the element takes up on the page. **The edge of the margin is invisible.**

Specifying Box Dimensions

- By default, the **width** and **height** of a block element is calculated automatically by browser (**the default auto value**).
- The box will be as wide as the browser window or other containing block element, and as tall as necessary to fit the content.
- Set the **width** and **height** properties to make the content area of an element a specific **width** or **height**.
- You can specify the width and height only for block-level elements and non-text inline elements such as images.
- The width and height properties do not apply to inline text elements (a, span, strong) and ignored by the browser.

box-sizing

- The **box-sizing** property defines how the **width** and **height** of an element are calculated: should they include padding and border, or not.

box-sizing

Values: **content-box** | border-box

Default: content-box

Applies to: all elements

Inherits: no

width, height

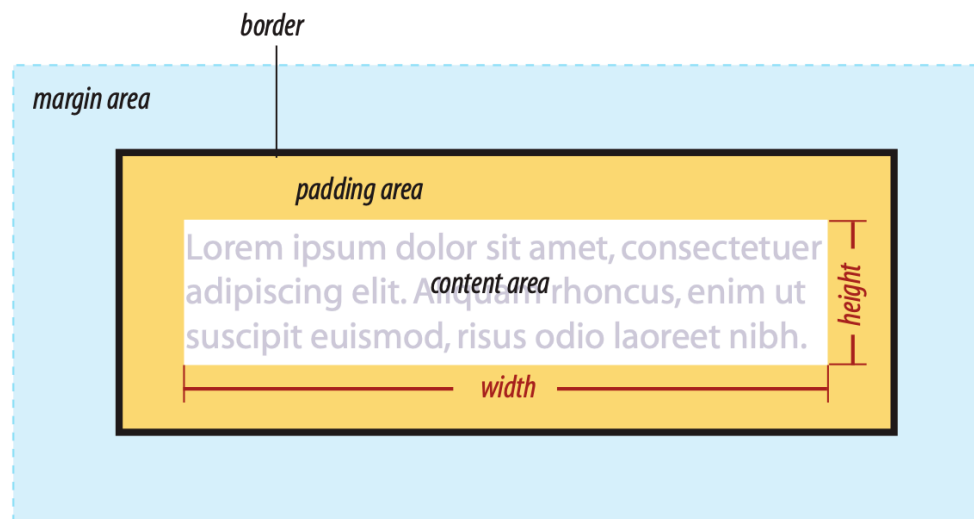
Values: length measurement | percentage | **auto** | inherit

Default: auto

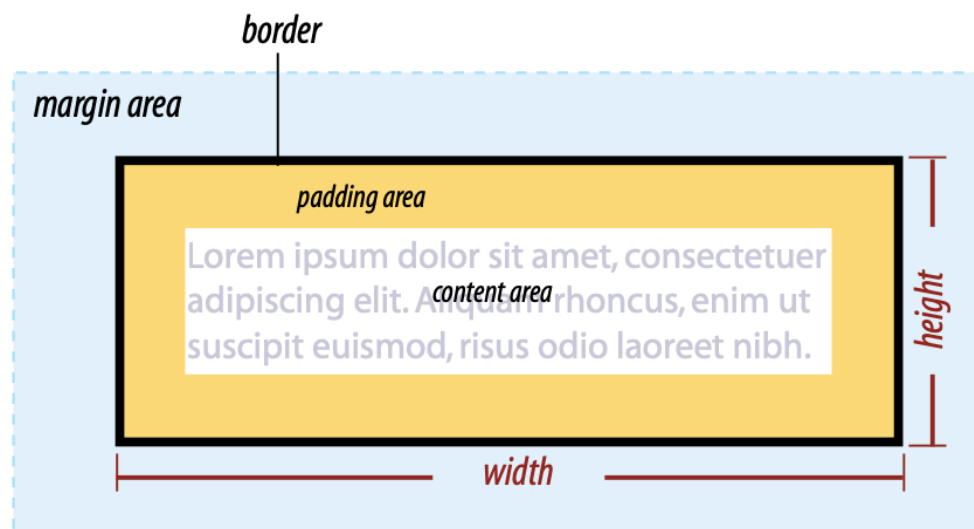
Applies to: block-level elements and replace inline elements (such as images)

Inherits: no

width, height



box-sizing: content-box



box-sizing: border-box

- Many developers find the **border-box** model to be more intuitive way to size elements.

content-box vs. border-box

```
box-sizing: content-box;  
width: 500px;
```

This week I am *extremely* excited about a new cooking technique called *sous vide*. In *sous vide* cooking, you submerge the food (usually vacuum-sealed in plastic) into a water bath that is precisely set to the target temperature you vacuum-sealed in plastic) into a water bath that is precisely set to the target temperature you want the food to be cooked to.

```
box-sizing: border-box;  
width: 500px;
```

Specifying Height

- The height property works just the same as width.
- In general, it is less common to specify the height of elements.
- It is best practice to let the height be calculated automatically by the browser.
- The default value of height property is **auto** that allows the element to grow or shrink based on its content (font size, user settings and screen scaling).
- Setting the height property can cause "**Overflow**" issues if the content is too long.

overflow

- To specify what to do with the content that doesn't fit

overflow

Values: [visible](#) | hidden | scroll | auto | inherit

Default: visible

Applies to: block-level elements and replace inline elements (such as images)

Inherits: no

Overflow

visible

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely cover the remainder of the glass in heavy paper. Any exposed areas (even inside) will get scratched by the flying sand, so it has to be a good seal.

hidden

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely cover the remainder of the glass

scroll

labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely cover the remainder of the glass in heavy paper. Any exposed areas (even

auto (short text)

Applying the masks to the glasses is the most labor-intensive part of the process.

auto (long text)

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely cover the remainder of the glass

Padding

- **Padding** is the space between the **content area** and the **border**.
- Adding padding around elements with background colors or borders creates space and prevents text from touching the edges.
- This extra spacing improves readability and gives the content a more balanced, comfortable appearance.

padding

padding-top, padding-right, padding-bottom, padding-left

Values: length measurement | percentage | inherit

Default: 0

Applies to: all elements except table-row-group, table-header-group, table-footer-group, table-column, and table-column-group

Inherits: no

padding (shorthand)

Values: length measurement | percentage | inherit

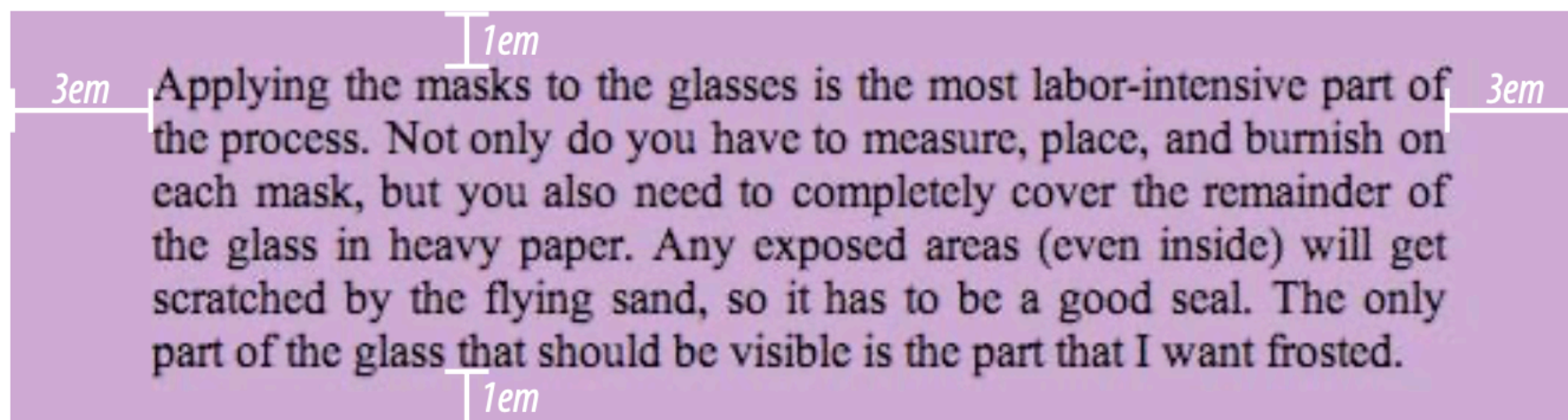
Default: 0

Applies to: all elements

Inherits: no

padding

```
blockquote {  
  padding-top: 1em;  
  padding-right: 3em;  
  padding-bottom: 1em;  
  padding-left: 3em;  
  background-color: #D098D4;  
}
```



padding (shorthand)

padding: 10px ; /* applied all sides */

padding: 10px 6px ; /* first => top, bottom second=> left, right*/

padding: 10px 6px 4px ; /* first => top

second=> left, right

third is bottom. */

padding: 10px 6px 4px 10px; /* top, right, bottom, left */

Borders

- **Border** is a simple a line drawn around the **content area** and its **padding**

border-top-style, border-right-style, border-bottom-style, border-left-style

Values: **none** | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit

Default: none (**The value hidden is equivalent to none.**)

Applies to: all elements

Inherits: no

border-style

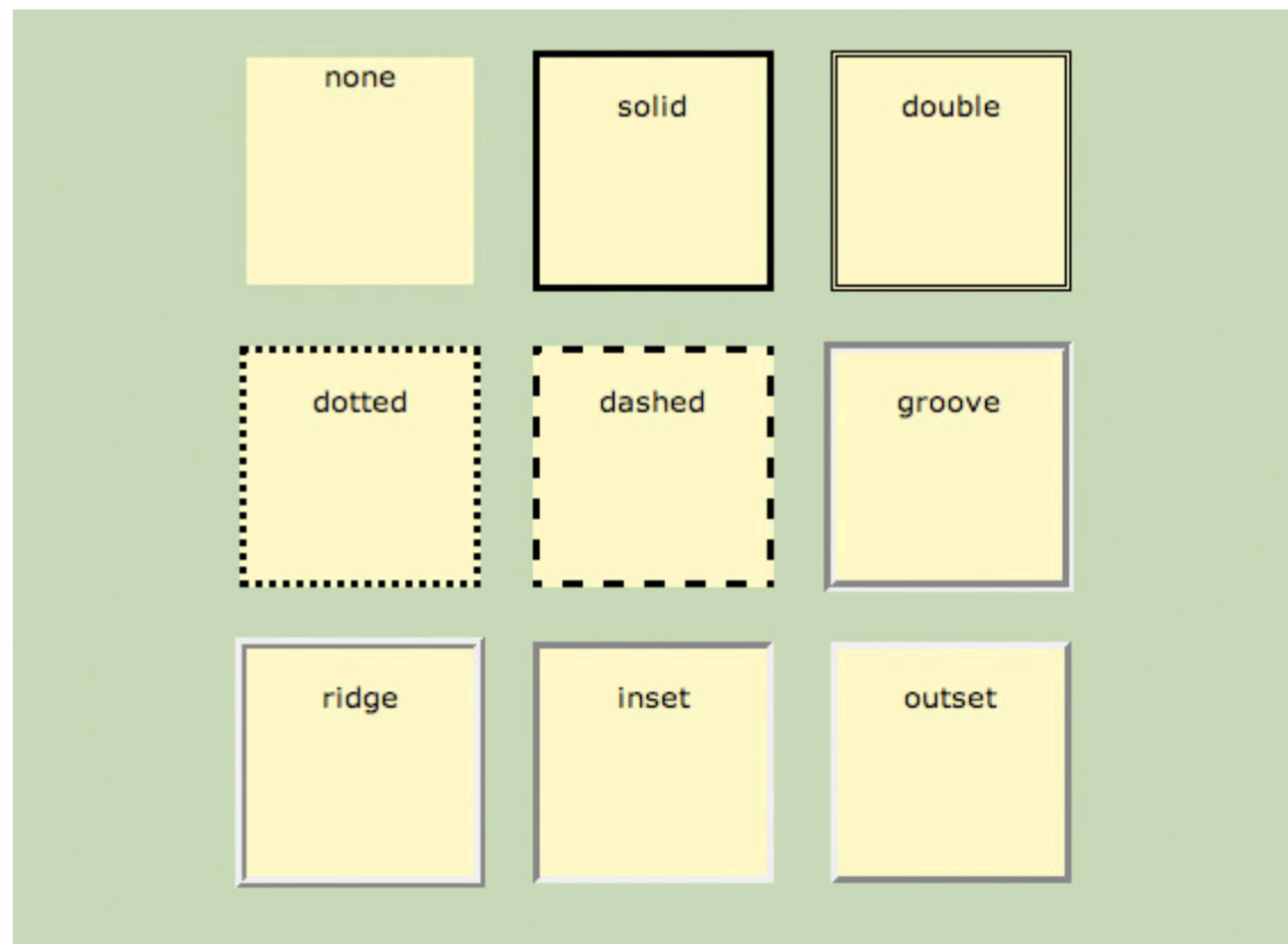
Values: **none** | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit

Default: none

Applies to: all elements

Inherits: no

border-style



Border width (thickness)

- Set the **border** width properties to specify the thickness of the **border**.

border-top-width, border-right-width, border-bottom-width, border-left-width

Values: length units | thin | **medium** | thick | inherit

Default: medium

Applies to: all elements

Inherits: no

border-width (shorthand)

Values: length units | thin | **medium** | thick | inherit

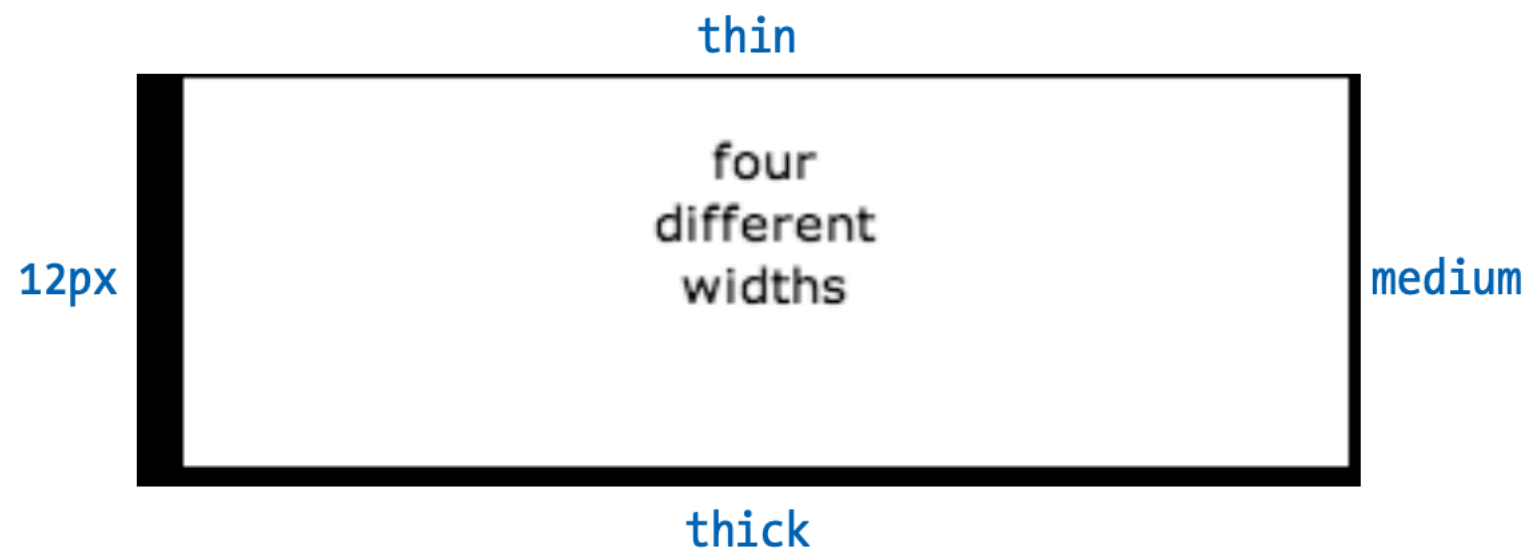
Default: medium

Applies to: all elements

Inherits: no

border-width

```
border-width: thin medium thick 12px ;  
/* top right bottom left */
```



Border color

`border-top-color, border-right-color, border-bottom-color, border-left-color`

Values: color name or RGB value | transparent | inherit

Default: the value of color property for the element

Applies to: all elements

Inherits: no

`border-color`

Values: color name or RGB value | transparent | inherit

Default: the value of color property for the element

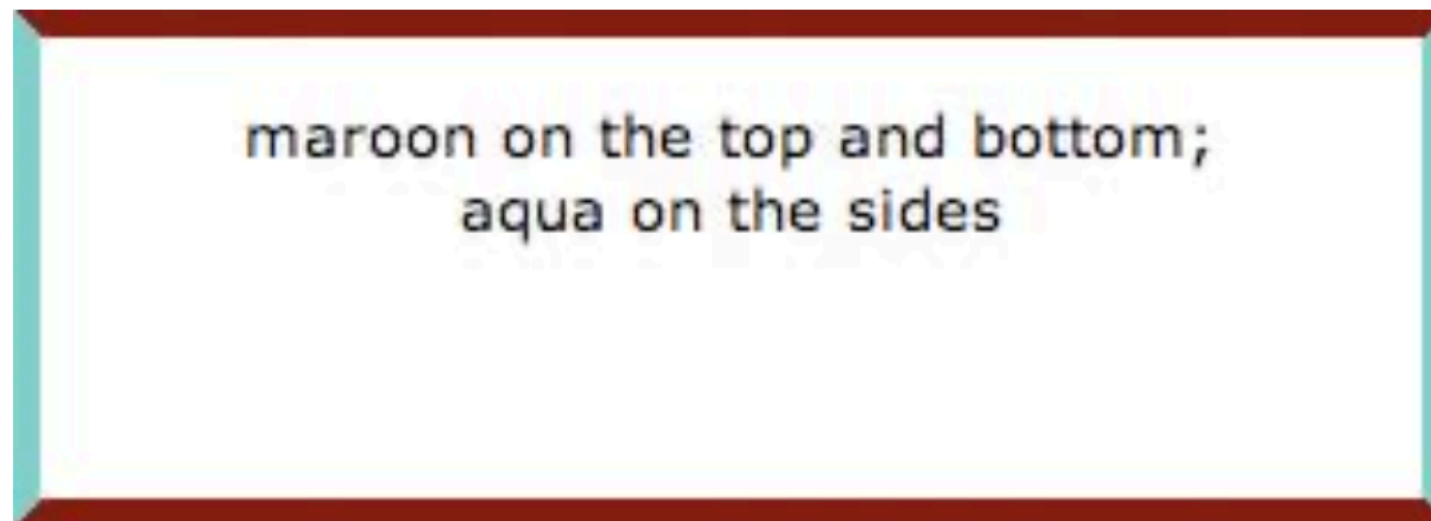
Applies to: all elements

Inherits: no

Border color

border-color: maroon aqua ;

/* first => top bottom
second => left right */



Combining style, width, and color

`border-top, border-right, border-bottom border-left`

Values: `border-style border-width border-color | inherit`

Default: `defaults for each property`

Applies to: `all elements`

Inherits: `no`

`border`

Values: `border-style border-width border-color | inherit`

Default: `defaults for each property`

Applies to: `all elements`

Inherits: `no`

Combining style, width, and color

```
h1 { border-left: red .5em solid; } /* left border only */
```

```
h2 { border-bottom: 1px solid; } /* bottom border only */
```

```
p.example { border: 2px dotted #663; } /* all four sides */
```

Rounded Corners with border-radius

- The border-radius property defines the radius of the element's corners. This property allows to add rounded corners to elements.

border-top-left-radius, border-top-right-radius, border-bottom-right-radius, border-bottom-left-radius

Values: length | percentage

Default: 0

Applies to: all elements

Inherits: no

border-radius

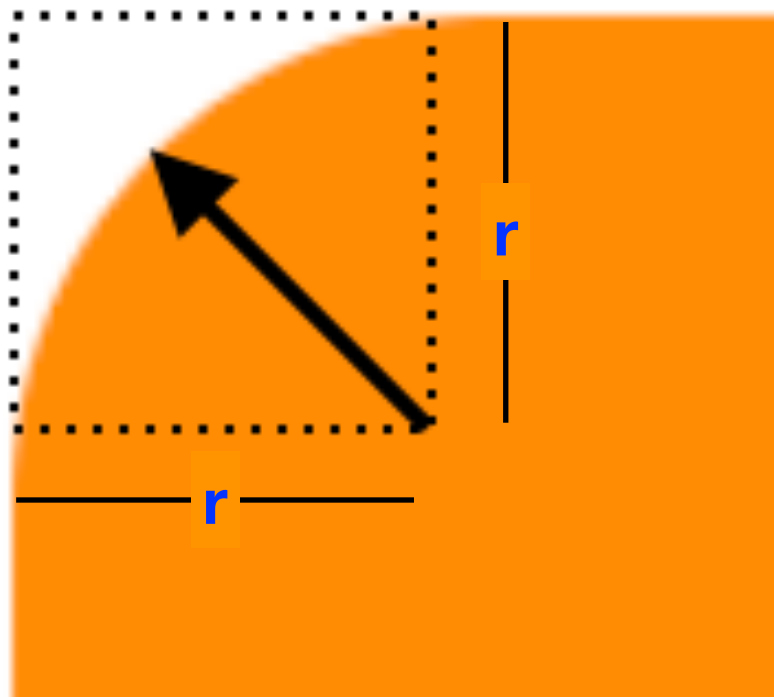
Values: 1,2,3 or 4 length or percentage values

Default: 0

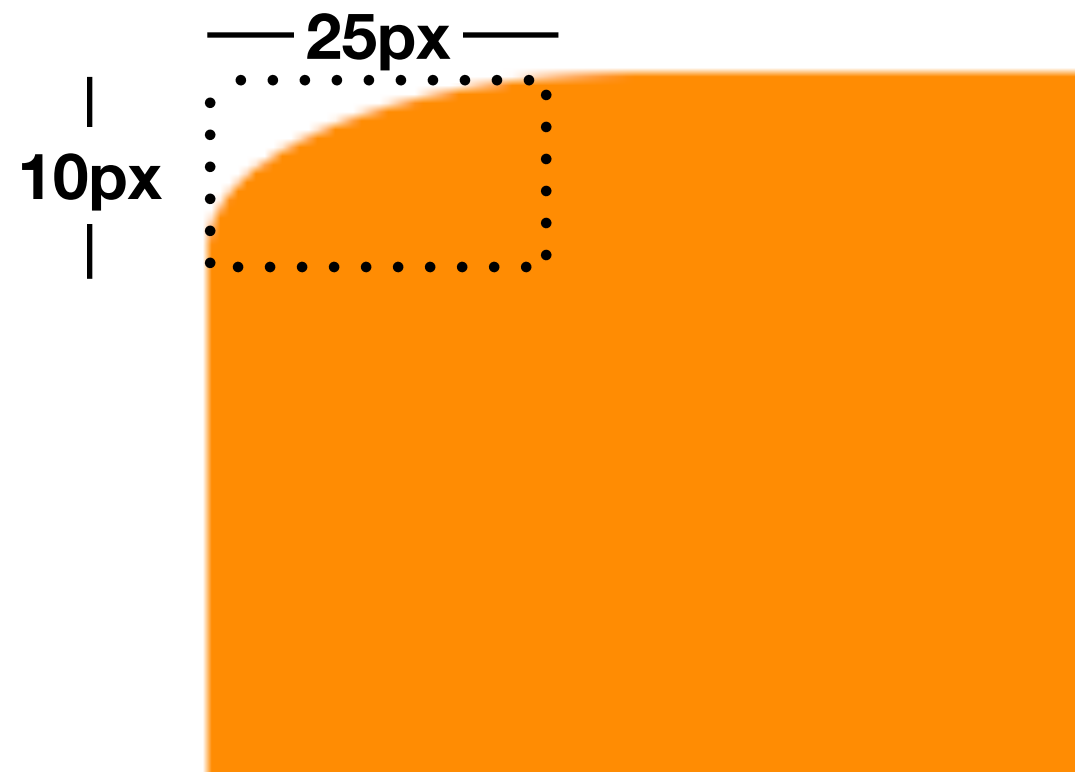
Applies to: all elements

Inherits: no

border-radius



border-radius: 25px ;



border-radius: 25px / 10px;

border-radius

```
p {  
  width: 200px;  
  height: 100px;  
  background: darkorange;  
}
```



`border-radius: 1em;`



`border-radius: 50px;`

```
border-radius: 1em; /* all four corners */  
border-radius: 1em 50px; /* top-left and bottom-right */  
border-radius: 50px 1em; /* top-right and bottom-left */
```



`border-top-right-radius: 50px;`



```
border-top-left-radius: 1em;  
border-top-right-radius: 2em;  
border-bottom-right-radius: 1em;  
border-bottom-left-radius: 2em;
```

Elliptical corners

- `border-top-right-radius : 100px      50px ;`
 horizontal radius vertical radius

A



`border-top-right-radius: 100px 50px;`

B



`border-top-right-radius: 50px 20px;`
`border-top-left-radius: 50px 20px;`

C



`border-radius: 60px / 40px;`

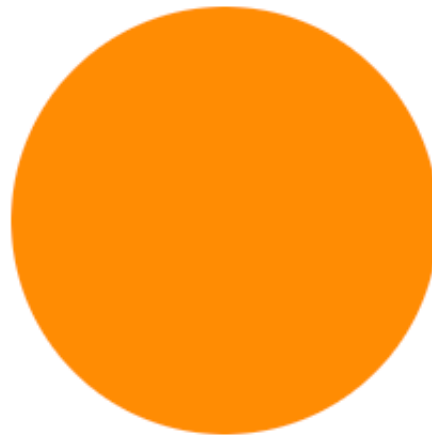
D



`border-radius:`
`36px 40px 60px 20px/12px 10px 30px 36px;`

Creating a Perfect Circle

```
.circle-element {  
  width: 100px;  
  height: 100px;  
  border-radius: 50%; /* Creates a perfect circle */  
}
```



Margins

- **Margin** is an optional amount of space that is added on the outside of the **border**.
- Margins keep elements from bumping into one another or the edge of the browser window or viewport.

margin-top, margin-right, margin-bottom, margin-left

Values: length measurement | percentage | auto | inherit

Default: auto

Applies to: all elements

Inherits: no

margin

Values: length measurement | percentage | auto | inherit

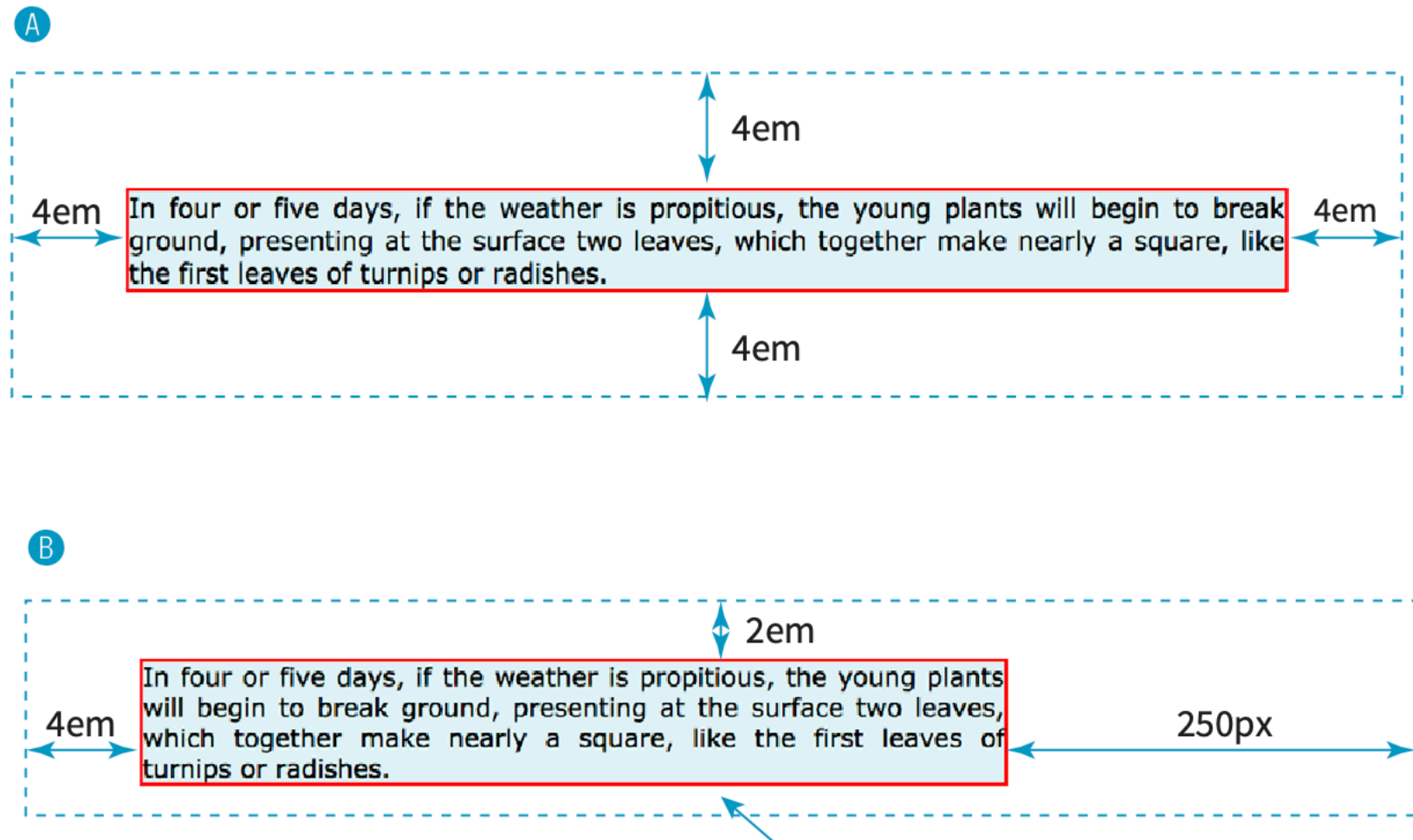
Default: auto

Applies to: all elements except elements with table display types other than table-caption, table, and inline-table

Inherits: no

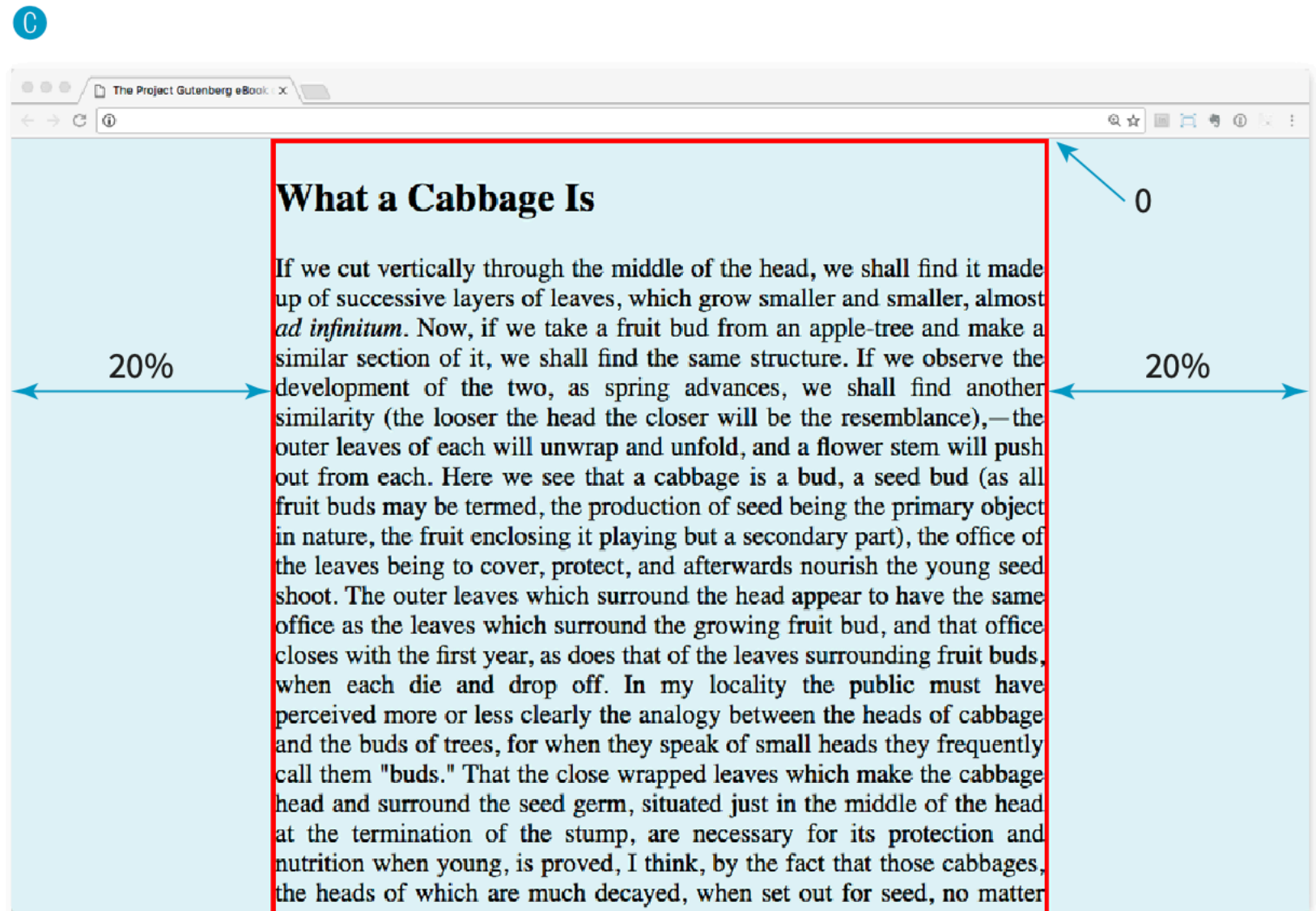
Margins

- A
- ```
p#A {
 margin: 4em;
 border: 2px solid red;
 background: #e2f3f5;
}
```
- B
- ```
p#B {  
  margin-top: 2em;  
  margin-right: 250px;  
  margin-bottom: 1em;  
  margin-left: 4em;  
  border: 2px solid red;  
  background: #e2f3f5;  
}
```



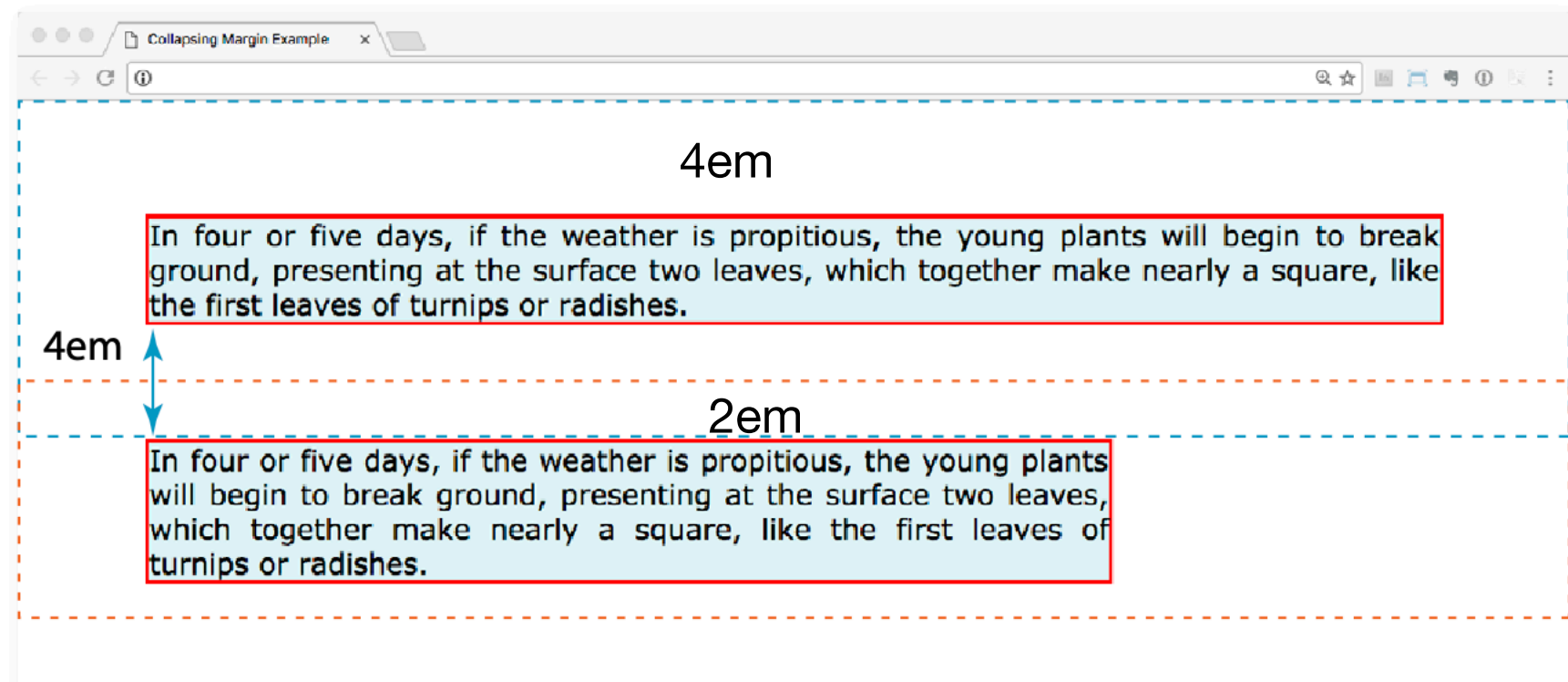
Margins

```
body {  
  margin: 0 20%;  
  border: 3px solid red;  
  background-color: #e2f3f5;  
}
```



Collapsing margins

- The most significant margin behavior to be aware of is the top and bottom margins of neighboring elements collapse.
- Vertical margins of neighboring elements collapse so that **only larger value is used** (not accumulating margins).



Margins on inline elements

- Applying top and bottom margins to inline elements will not add vertical space above or below the element, and the height of the line will not change.
- However, applying left and right margins to inline text elements, margin space will held clear before and after the text.
- Margins on replaced inline elements (such as images), do render on all sides and do affect the height of the line.

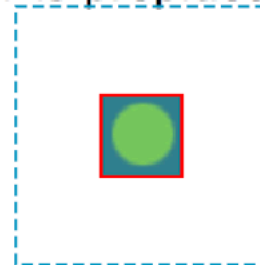
Margins on inline elements

```
em { margin: 2em; }
```

In four or five days, if the weather is propitious, the young plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

```
img { margin: 2em; }
```

In four or five days, if the weather is propitious, the young



plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

Negative margins

- When applying a negative margin, the content, padding and border are moved in the opposite direction.

```
p.top { margin-bottom: 3em; }
```

Pushes the following paragraph down by 3 ems.

In four or five days, if the weather is propitious, the young plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

In four or five days, if the weather is propitious, the young plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

```
p.top { margin-bottom: -3em; }
```

The following element moves up by 3 ems.

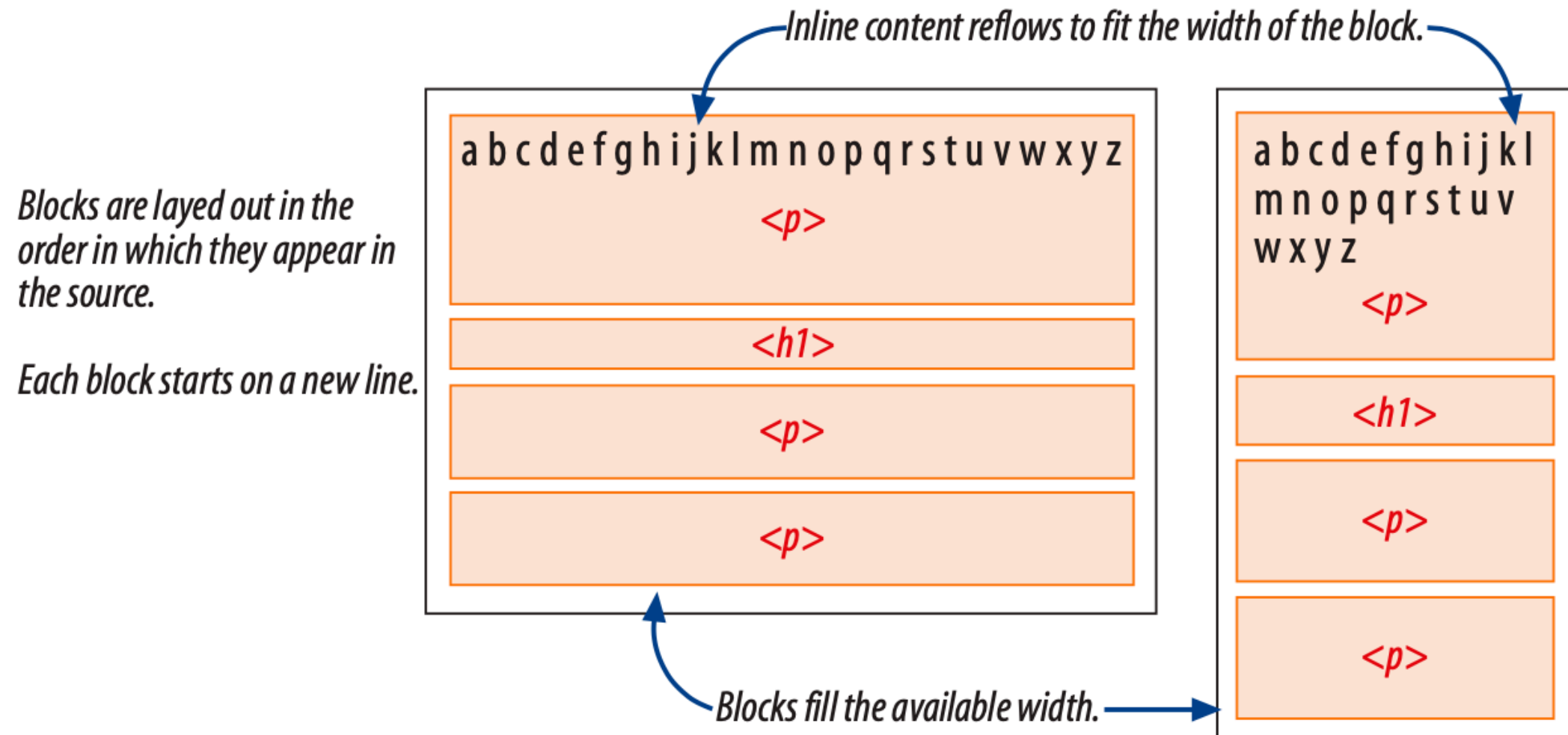
In four or five days, if the weather is propitious, the young plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

In four or five days, if the weather is propitious, the young plants will begin to break ground, presenting at the surface two leaves, which together make nearly a square, like the first leaves of turnips or radishes.

Normal Flow

- In the **CSS layout** model, **text elements** are laid out from **top to bottom** and from **left to right** (in left-to-right reading language).
- **Block elements** stack up on top of one another and fill the available width of the browser window or other containing element.
- **Inline elements** and **text characters** line up next to one another to fill the block elements.
- When the window or containing element is **resized**
 - The **block elements** expand or contract to the new width
 - The **inline content** reflows to fit the block element.

Normal flow



Floating and Positioning

- **Floating** and **Positioning** are the CSS methods for breaking out of the normal flow and arranging elements on the page.
- **Floating** an element moves it to the left or right, and allows the following text to wrap around it.
- **Positioning** is a way to specify the location of an element anywhere on the page with pixel precision.

Floating

- The `float` property moves an element as far as possible to the left or right, allowing the following content to wrap around it.

`float`

Values: left | right | none | inherit

Default: none

Applies to: all elements

Inherits: no

Floating

The markup

```
<p>They went down, down,...</p>
```

The style sheet

```
img {  
  float: right;  
}  
p {  
  padding: 15px;  
  background-color: #FFF799;  
  border: 2px solid #6C4788;  
}
```

```
img {  
  float: right;  
  margin: 10px ;  
}
```

Inline image in the normal flow

space next to the image is held clear



They went down, down, down, till at last they came to a passage with a door at one end, which was only fastened with a latch. The eldest Princess opened it, and they found themselves immediately in a lovely little wood, where the leaves were spangled with drops of silver which shone in the brilliant light of the moon. They next crossed another wood where the leaves were sprinkled with gold, and after that another still, where the leaves glittered with diamonds.

Inline image floated to the right.

They went down, down, down, till at last they came to a passage with a door at one end, which was only fastened with a latch. The eldest Princess opened it, and they found themselves immediately in a lovely little wood, where the leaves were spangled with drops of silver which shone in the brilliant light of the moon. They next crossed another wood where the leaves were sprinkled with gold, and after that another still, where the leaves glittered with diamonds.



image moves over and text wraps around it

Indicates outer margin edge
(this rule would not appear in the actual web page)

They went down, down, down, till at last they came to a passage with a door at one end, which was only fastened with a latch. The eldest Princess opened it, and they found themselves immediately in a lovely little wood, where the leaves were spangled with drops of silver which shone in the brilliant light of the moon. They next crossed another wood where the leaves were sprinkled with gold, and after that another still, where the leaves glittered with diamonds.



Key behaviors of floated elements

- A floated element is not flow. Only the non-floated element has to flow around it (like an Island in a stream).
- Floats stay in the content area of the containing element.
- Margin are maintained. The margins are held on all sides of the floated image. The entire element box, from outer edge to outer edge, is floated.
- It is possible to float any HTML element, both inline and block-level.

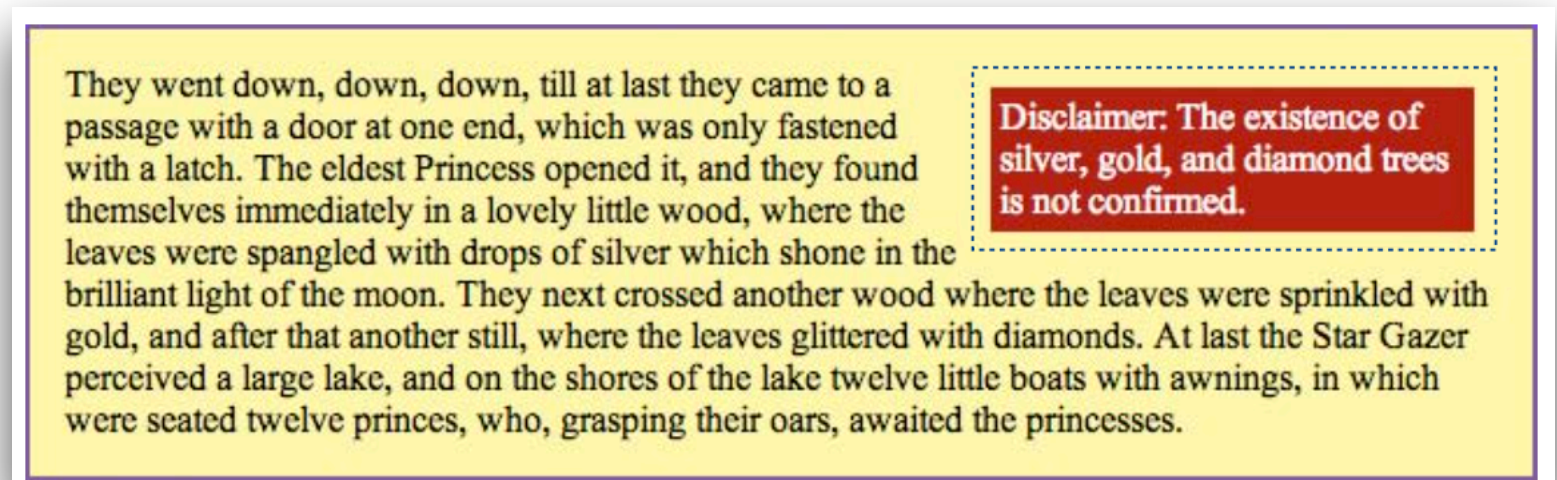
Floating inline text elements

The markup

```
<p><span class="disclaimer">Disclaimer: The existence of silver, gold, and diamond trees is not confirmed.</span> They went down, down, down, till at last they came to a passage... </p>
```

The style sheet

```
span.disclaimer {  
    float: right;  
    margin: 10px;  
    width: 200px;  
    color: #FFF;  
    background-color: #9D080D;  
    padding: 4px;  
}  
p {  
    padding: 15px;  
    background-color: #FFF799;  
    border: 2px solid #6C4788;  
}
```



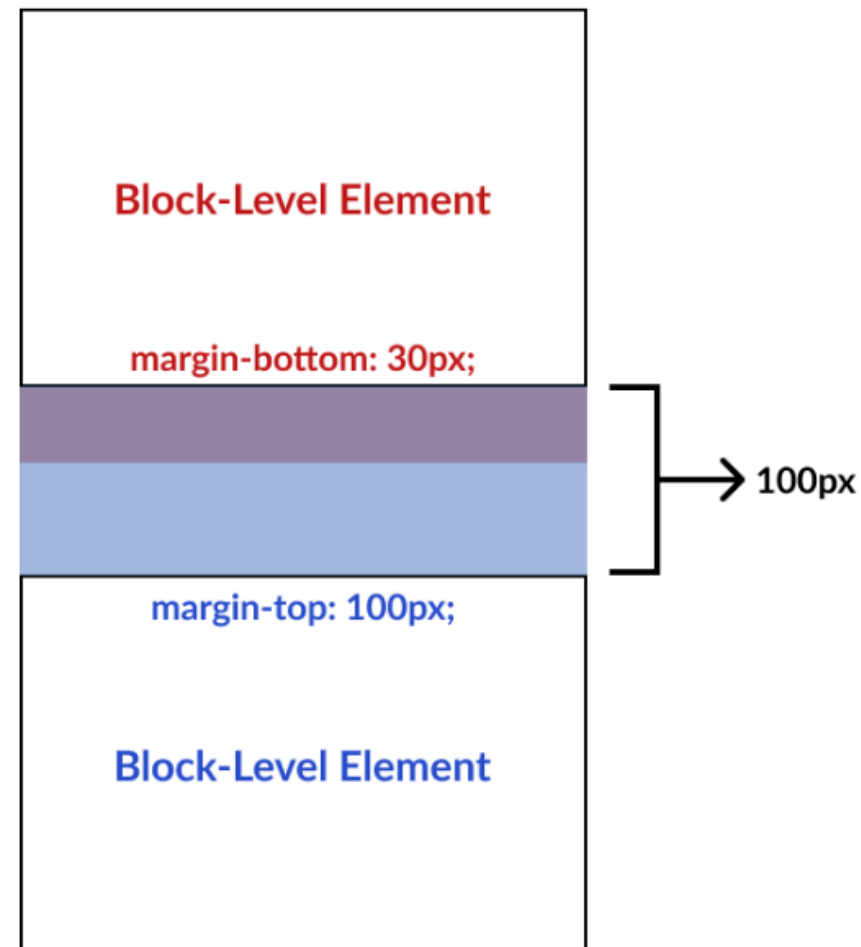
Key behaviors of floated text elements

- Always provide a **width** for floated text elements
- Floated inline elements behave as **block elements**
- **Margin** on floated elements do not collapse
 - In the normal flow, **vertical margins of adjacent elements (top and bottom margin) are collapse (overlap)** but **margins for floated elements are maintained on all sides as specified.**

Margin collapse

```
<div class="one">Block-Level Element One</div>  
<div class="two">Block-level Element Two</div>
```

```
div.one {  
  margin-bottom: 30px;  
}  
  
div.two {  
  margin-top: 100px;  
}
```



- Instead of $30\text{px} + 100\text{px} = 130\text{px}$, the margin between `.one` and `.two` is 100px (the larger value).

[\[Link\]](#)

Floating block elements

The markup

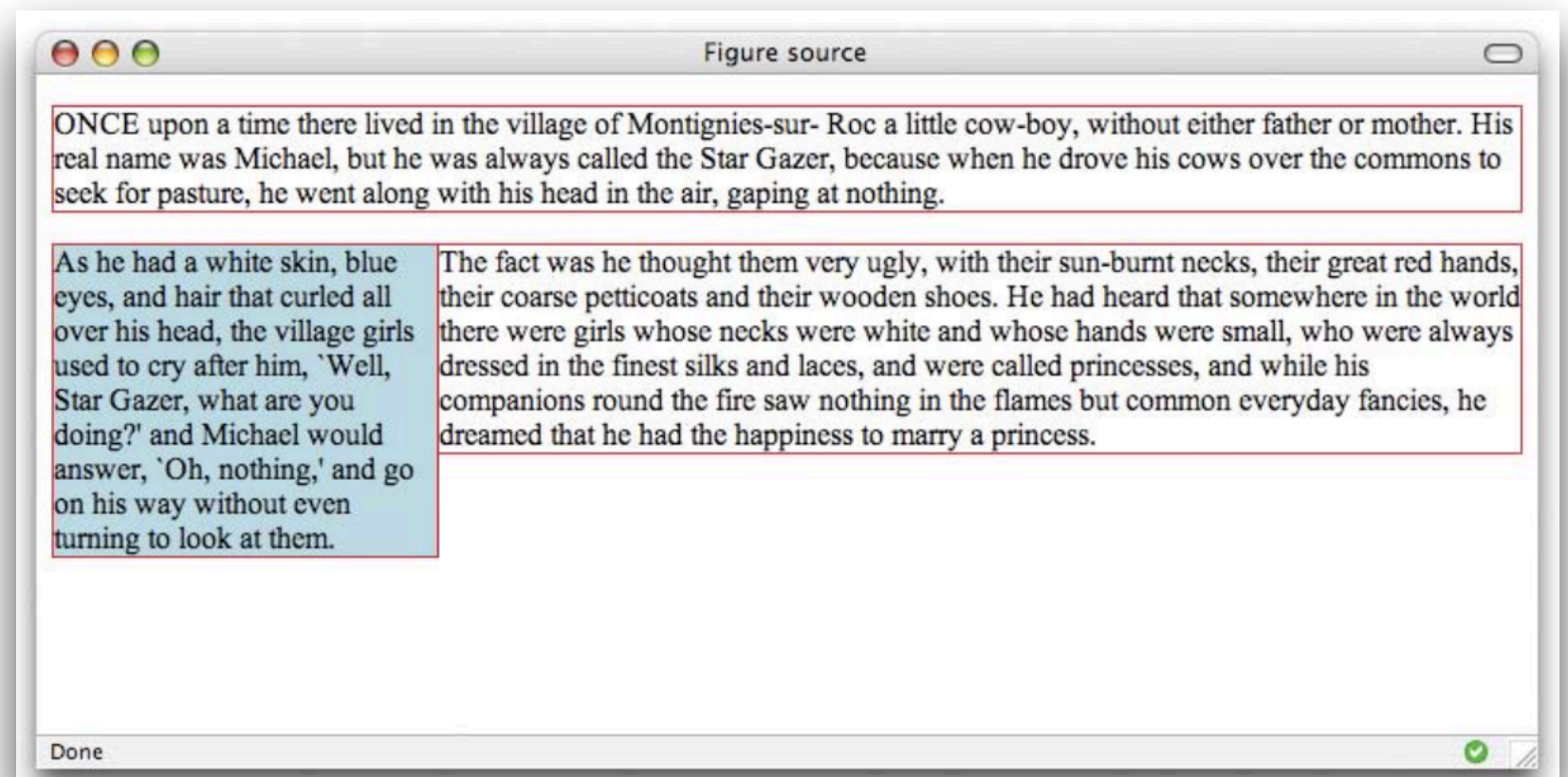
```
<p>ONCE upon a time....</p>
```

```
<p id="float">As he had a white skin, blue eyes,...</p>
```

```
<p>The fact was he thought them very ugly...</p>
```

The style sheet

```
p#float {  
    float: left;  
    width: 200px;  
    margin-top: 0px;  
    background: #A5D3DE;  
}  
p {  
    border: 1px solid red;  
}
```



Key points of floated block elements

- Always provide a **width** for floated block elements
- Elements do not float higher than their reference in the source.
- Non-floated elements maintain the normal flow.
 - Adding a left margin to the surrounding paragraphs would add space on the left edge of the page, not between the text and the floated element.
 - If you want space between the float and the wrapped text, you need to apply the margin to the float itself.

Add margin to floated elements

Alice, a young girl, sits bored by a riverbank and spots a White Rabbit with a pocket watch and waistcoat lamenting that he is late. Surprised, Alice follows him down a rabbit hole, which sends her into a lengthy plummet but to a safe landing. Inside a room with a table, she finds a key to a tiny door, beyond which is a garden.

While pondering how to fit through the door, she discovers a bottle labelled "Drink me". Alice drinks some of the bottle's contents, and to her astonishment, she shrinks small enough to enter the door. However, she had left the key upon the table and cannot reach it.

Alice then discovers and eats a cake labelled "Eat me", which causes her to grow to a tremendous size. Unhappy, Alice bursts into tears, and the passing White Rabbit flees in a panic, dropping a fan and two gloves. Alice uses the fan for herself, which causes her to shrink once more and leaves her swimming in a pool of her own tears. Within the pool, Alice meets various animals and birds, who convene on a bank and engage in a "Caucus Race" to dry themselves. Following the end of the race, Alice inadvertently frightens the animals away by discussing her cat.

Alice, a young girl, sits bored by a riverbank and spots a White Rabbit with a pocket watch and waistcoat lamenting that he is late. Surprised, Alice follows him down a rabbit hole, which sends her into a lengthy plummet but to a safe landing. Inside a room with a table, she finds a key to a tiny door, beyond which is a garden.

While pondering how to fit through the door, she discovers a bottle labelled "Drink me". Alice drinks some of the bottle's contents, and to her astonishment, she shrinks small enough to enter the door. However, she had left the key upon the table and cannot reach it.

various animals and birds, who convene on a bank and engage in a "Caucus Race" to dry themselves. Following the end of the race, Alice inadvertently frightens the animals away by discussing her cat.

Alice then discovers and eats a cake labelled "Eat me", which causes her to grow to a tremendous size. Unhappy, Alice bursts into tears, and the passing White Rabbit flees in a panic, dropping a fan and two gloves. Alice uses the fan for herself, which causes her to shrink once more and leaves her swimming in a pool of her own tears. Within the pool, Alice meets

```
.float {
  float: left;
  width: 300px ;
}
.unfloat {
  margin: 20px ;
  border: 1px red solid;
}
```

Alice, a young girl, sits bored by a riverbank and spots a White Rabbit with a pocket watch and waistcoat lamenting that he is late. Surprised, Alice follows him down a rabbit hole, which sends her into a lengthy plummet but to a safe landing. Inside a room with a table, she finds a key to a tiny door, beyond which is a garden.

While pondering how to fit through the door, she discovers a bottle labelled "Drink me". Alice drinks some of the bottle's contents, and to her astonishment, she shrinks small enough to enter the door. However, she had left the key upon the table and cannot reach it.

Alice then discovers and eats a cake labelled "Eat me", which causes her to grow to a tremendous size. Unhappy, Alice bursts into tears, and the passing White Rabbit flees in a panic, dropping a fan and two gloves. Alice uses the fan for herself, which causes her to shrink once more and leaves her swimming in a pool of her own tears. Within the pool, Alice meets various animals and birds, who convene on a bank and engage in a "Caucus Race" to dry themselves. Following the end of the race, Alice inadvertently frightens the animals away by discussing her cat.

```
.float {
  float: left;
  width: 300px ;
  margin: 10px ;
  border: 1px red solid ;
}
.unfloat {
}
```

[Demo]

Assigning Display Roles

- The **Display** property
 - is the most important **CSS** property for controlling layout.
 - specifies how an element is displayed.
- Every **HTML element** has a default **display** value depending on what type of element it is.
- The default display value for most element is **block** or **inline**. However, this property can be overridden.

display

display

Values:	inline block list-item inline-block table inline-table table-row-group table-header-group table-footer-group table-row table-column-group table-column table-cell table-caption none
---------	--

Default:	inline
----------	--------

Applies to:	all elements
-------------	--------------

Inherits:	yes
-----------	-----

```
li {  
  display: inline ;  
}  
span {  
  display: block ;  
}
```

display

- It is common practice to make li elements **display** as **inline** elements to turn a list into a horizontal navigation bar.
- Make an **inline** a (anchor) element display as a **block** in order to give it a specific width and height.
- Set the **display** property to none, which removes the content (and space) from the normal flow entirely.
- Unlike **visibility: hidden**, which just makes the element invisible but holds the space it would have occupied.

Clearing floated elements

- Set the `clear` property to turn the text wrapping off and get back to layout as usual.
- Applying the `clear` property to an element to start below the floated element, not the floated element itself.

`clear`

Values: left | right | both | none | inherit

Default: none

Applies to: block-level elements only

Inherits: no

No clear

The markup

```
<p class="float">This is a paragraph that floats on the left</p>  
<p>This is another paragraph that follows the floated element</p>
```

The style sheet

```
p.float {  
    float: left;  
    width: 100px;  
    margin-top: 0; /* keep text to show in the same line */  
    border: 2px dotted red  
}  
p.clear {  
    clear: left ;  
}
```

This is a
paragraph that
floats on the
left

This is another paragraph that follows the floated element

Demo

clear

The markup

```
<p class="float">This is a paragraph that floats on the left</p>  
<p class="clear">This is another paragraph that follows the floated element</p>
```

The style sheet

```
p.float {  
    float: left;  
    width: 100px;  
    margin-top: 0; /* keep text to show in the same line */  
    border: 2px dotted red  
}  
p.clear {  
    clear: left ;  
}
```

This is a
paragraph that
floats on the
left

This is another paragraph that follows the floated element

Demo

Using floats to create columns

- For a two-column float
 - Float **one div** and add **a wide margin** on the side of the text element that wraps around it.
 - Float both **divs** to **left** or **right**.
 - Float one **div** to the **left** and the second **div** to the **right** (or vice versa).

Float one div

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style>
      .left {
        margin-right: 10px ;
        float: left ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
      .right {
        margin-right: 10px ;
        float: right ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
    </style>
  </head>
  <body>
    <div class="left">In HTML, when we wish to separate our page
      into different columns, there are multiple
      ways to do achieve this in CSS. One of the
      simpler methods is to set the display property
      to inline-block. Let's see how with a code example.</div>
    <div>The reason for this is that there's a carriage return
      between these two div elements due to them being in
      separate lines in the HTML code. The browser parses
      this carriage return (or newline) as a space character
      and adds it between the two divs. Hence, the second div
      (which occupies 66.66% of the screen width) is unable to
      find the necessary space to accommodate itself and runs
      over to the next line.</div>
  </body>
</html>
```

In HTML, when we wish to separate our page into different columns, there are multiple ways to do achieve this in CSS. One of the simpler methods is to set the display property to inline-block. Let's see how with a code example.

The reason for this is that there's a carriage return between these two div elements due to them being in separate lines in the HTML code. The browser parses this carriage return (or newline) as a space character and adds it between the two divs. Hence, the second div (which occupies 66.66% of the screen width) is unable to find the necessary space to accommodate itself and runs over to the next line.

Float both divs

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style>
      .left {
        margin-right: 10px ;
        float: left ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
      .right {
        margin-right: 10px ;
        float: right ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
    </style>
  </head>
  <body>
    <div class="left">In HTML, when we wish to separate our page
      into different columns, there are multiple
      ways to do achieve this in CSS. One of the
      simpler methods is to set the display property
      to inline-block. Let's see how with a code example.</div>
    <div class="left">The reason for this is that there's a
      carriage return between these two div elements due to
      them being in separate lines in the HTML code. The browser
      parses this carriage return (or newline) as a space character
      and adds it between the two divs. Hence, the second div
      (which occupies 66.66% of the screen width) is unable to
      find the necessary space to accommodate itself and runs
      over to the next line.</div>
  </body>
</html>
```

In HTML, when we wish to separate our page into different columns, there are multiple ways to do achieve this in CSS. One of the simpler methods is to set the display property to inline-block. Let's see how with a code example.

The reason for this is that there's a carriage return between these two div elements due to them being in separate lines in the HTML code. The browser parses this carriage return (or newline) as a space character and adds it between the two divs. Hence, the second div (which occupies 66.66% of the screen width) is unable to find the necessary space to accommodate itself and runs over to the next line.

Float one div to left, one to right

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style>
      .left {
        margin-right: 10px ;
        float: left ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
      .right {
        margin-right: 10px ;
        float: right ;
        width: 48% ;
        background-color: yellow ;
        text-align: justify ;
      }
    </style>
  </head>
  <body>
    <div class="left">In HTML, when we wish to separate our page
      into different columns, there are multiple
      ways to do achieve this in CSS. One of the
      simpler methods is to set the display property
      to inline-block. Let's see how with a code example.</div>
    <div class="right">The reason for this is that there's a
      carriage return between these two div elements due to
      them being in separate lines in the HTML code. The browser
      parses this carriage return (or newline) as a space character
      and adds it between the two divs. Hence, the second div
      (which occupies 66.66% of the screen width) is unable to
      find the necessary space to accommodate itself and runs
      over to the next line.</div>
  </body>
</html>
```

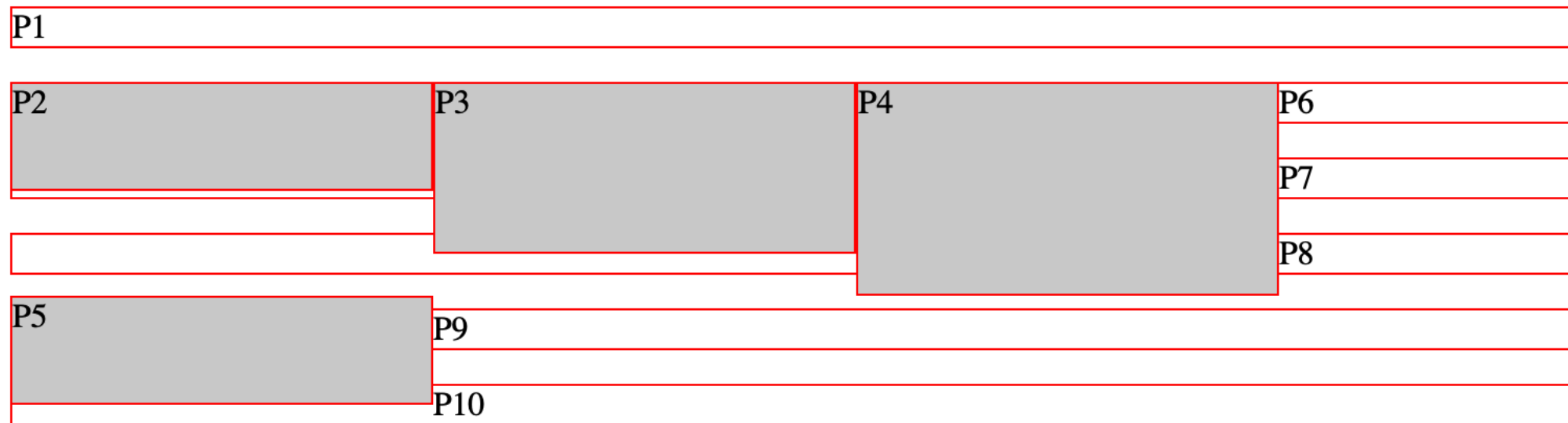
In HTML, when we wish to separate our page into different columns, there are multiple ways to do achieve this in CSS. One of the simpler methods is to set the display property to inline-block. Let's see how with a code example.

The reason for this is that there's a carriage return between these two div elements due to them being in separate lines in the HTML code. The browser parses this carriage return (or newline) as a space character and adds it between the two divs. Hence, the second div (which occupies 66.66% of the screen width) is unable to find the necessary space to accommodate itself and runs over to the next line.

Using floats to create columns

- Every **float** needs to have a specified **width**.
- Calculate the **widths** of each column correctly, factoring in padding, borders, and margins.
- If the **total width** of all columns **exceeds** the available **width** of the **browser** or other containing element, the **final floated column** will **drop down** below the column next to it.
- The **limitation** to using **floats** for **columns** is that it is **dependent** on the **order** of the **elements** in the source document.
- The **floated element** must appear **before** the content that wraps around it and the **source document** may **not** always be **ordered conveniently**.

Floating multiple element



<p>P1</p>

<p class="float2">P2</p>

<p class="float3">P3</p>

<p class="float4">P4</p>

<p class="float5">P5</p>

<p>P6</p>

<p>P7</p>

<p>P8</p>

<p>P9</p>

<p>P10</p>

Elements (P5) floated to the same side line up. if there is not enough room, subsequent elements move down.

Demo

Transform list to a tab-like menu



```
<ul>
  <li><a href="#">Serif</a></li>
  <li><a href="#">Sans-serif</a></li>
  <li><a href="#">Script</a></li>
  <li><a href="#">Display</a></li>
  <li><a href="#">Dingbats</a></li>
</ul>
```

```
<style>
  ul {
    list-style-type: none;
    margin: 0;
    padding: 0;
  }
  ul li {
    float: left ;
    padding: 10px ;
    margin: 5px ;
    background: yellow ;
    border-radius: 10px 10px 0 0;
  }
  ul li a {
    display: block ;
    text-decoration: none ;
  }
</style>
```

Demo

Containing Floats

- By default, floats are designed to hang out of the element they are contained in.
- That's just fine for allowing text to flow around a floated image, but some times it can cause unwanted behaviors.

Containing Float



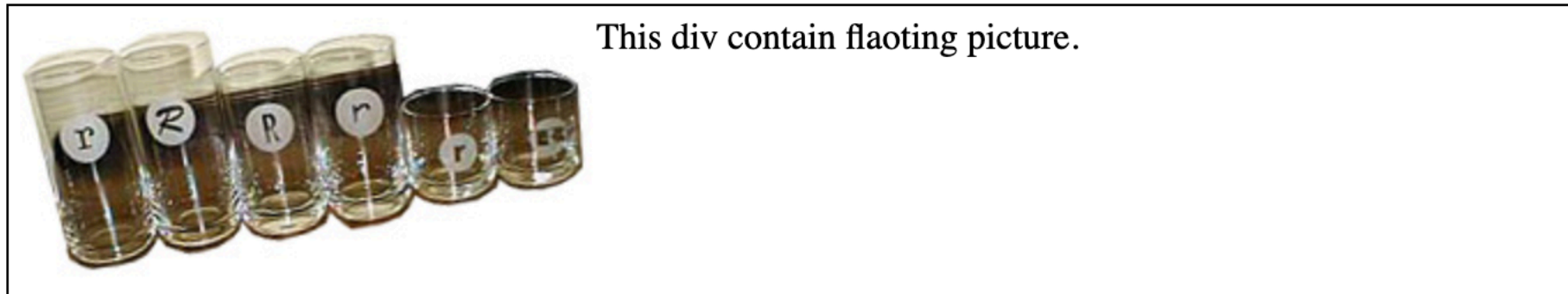
This div contain flaoing picture.

<> demo_contain_float.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Containing Float</title>
6      <style>
7          .pic {
8              float: left ;
9          }
10         div {
11             padding: 5px ;
12             border: 1px solid black ;
13         }
14     </style>
15 </head>
16 <body>
17     <div>
18          This div contain flaoing picture.
19     </div>
20 </body>
21 </html>
```

Demo

Set overflow



```
<> demo_contain_float.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Containing Float</title>
6      <style>
7          .pic {
8              float: left ;
9          }
10         div {
11             padding: 5px ;
12             border: 1px solid black ;
13             overflow: auto;
14         }
15     </style>
16 </head>
17 <body>
18     <div>
19          This div contain flaoting picture.
20     </div>
21 </body>
22 </html>
```

Demo

Container with float

What's an API?

An API is a set of definitions and protocols for building and integrating application software. It's sometimes referred to as a contract between an information provider and an information user—establishing the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user supply a zip code and that the producer reply with a 2-part answer, the first being the high temperature, and the second being the low.

In other words, if you want to interact with a computer or system to retrieve information or perform a function, an API helps you communicate what you want to that system so it can understand and fulfill the request.

```
6 <style>
7   .container {
8     border: 1px dotted blue;
9     background-color: #aliceblue;
10  }
11 </style>
12 </head>
13
14 <body>
15   <h1> What's an API?</h1>
16   <div class="container">
17     <p>
18       An API is a set of definitions and protocols for building and integrating application software.
19       It's sometimes referred to as a contract between an information provider and an information user—establishing
20       the content required from the consumer (the call) and the content required by the producer (the response). For example,
21       the API design for a weather service could specify that the user supply a zip code and that the producer reply
22       with a 2-part answer, the first being the high temperature, and the second being the low.
23     </p>
24
25     <p>
26       In other words, if you want to interact with a computer or system to retrieve information or perform a function, an
27       API helps you communicate what you want to that system so it can understand and fulfill the request.
28     </p>
29   </div>
30 </body>
31 </html>
```

Container with float

```
6  <style>
7    .container {
8      border: 1px dotted blue;
9      background-color: #aliceblue;
10   }
11   .float {
12     width: 40%;
13     float: left;
14   }
15 </style>
16 </head>
17
18 <body>
19   <h1> What's an API?</h1>
20   <div class="container">
21     <p class="float">
22       An API is a set of definitions and protocols for building and integrating application software.
23       It's sometimes referred to as a contract between an information provider and an information user—establishing
24       the content required from the consumer (the call) and the content required by the producer (the response). For example,
25       the API design for a weather service could specify that the user supply a zip code and that the producer reply
26       with a 2-part answer, the first being the high temperature, and the second being the low.
27     </p>
28
29     <p class="float">
30       In other words, if you want to interact with a computer or system to retrieve information or perform a function, an
31       API helps you communicate what you want to that system so it can understand and fulfill the request.
32     </p>
33   </div>
34 </body>
35 </html>
```

What's an API?

An API is a set of definitions and protocols for building and integrating application software. It's sometimes referred to as a contract between an information provider and an information user—establishing the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user supply a zip code and that the producer reply with a 2-part answer, the first being the high temperature, and the second being the low.

In other words, if you want to interact with a computer or system to retrieve information or perform a function, an API helps you communicate what you want to that system so it can understand and fulfill the request.

Set overflow

What's an API?

An API is a set of definitions and protocols for building and integrating application software. It's sometimes referred to as a contract between an information provider and an information user—establishing the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user supply a zip code and that the producer reply with a 2-part answer, the first being the high temperature, and the second being the low.

In other words, if you want to interact with a computer or system to retrieve information or perform a function, an API helps you communicate what you want to that system so it can understand and fulfill the request.

```
6 <style>
7   .container {
8     overflow: auto;
9     border: 1px dotted blue;
10    background-color: #aliceblue;
11  }
12  .float {
13    width: 40%;
14    float: left;
15  }
16 </style>
17 </head>
18
19 <body>
20   <h1> What's an API?</h1>
21   <div class="container">
22     <p class="float">
23       An API is a set of definitions and protocols for building and integrating application software.
24       It's sometimes referred to as a contract between an information provider and an information user—establishing
25       the content required from the consumer (the call) and the content required by the producer (the response). For example,
26       the API design for a weather service could specify that the user supply a zip code and that the producer reply
27       with a 2-part answer, the first being the high temperature, and the second being the low.
28     </p>
29
30     <p class="float">
31       In other words, if you want to interact with a computer or system to retrieve information or perform a function, an
32       API helps you communicate what you want to that system so it can understand and fulfill the request.
33     </p>
34   </div>
35 </body>
36 </html>
```

Demo

Set container to float

```
6  <style>
7    .container {
8      float: left ;
9      width: 100%;
10     border: 1px dotted blue;
11     background-color: #aliceblue;
12   }
13   .float {
14     width: 40% ;
15     float: left;
16   }
17 </style>
18 </head>
19
20 <body>
21   <h1> What's an API?</h1>
22   <div class="container">
23     <p class="float">
24       An API is a set of definitions and protocols for building and integrating application software.
25       It's sometimes referred to as a contract between an information provider and an information user—establishing
26       the content required from the consumer (the call) and the content required by the producer (the response). For example,
27       the API design for a weather service could specify that the user supply a zip code and that the producer reply
28       with a 2-part answer, the first being the high temperature, and the second being the low.
29     </p>
30
31     <p class="float">
32       In other words, if you want to interact with a computer or system to retrieve information or perform a function, an
33       API helps you communicate what you want to that system so it can understand and fulfill the request.
34     </p>
35   </div>
36 </body>
37 </html>
```

What's an API?

An API is a set of definitions and protocols for building and integrating application software. It's sometimes referred to as a contract between an information provider and an information user—establishing the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user supply a zip code and that the producer reply with a 2-part answer, the first being the high temperature, and the second being the low.

In other words, if you want to interact with a computer or system to retrieve information or perform a function, an API helps you communicate what you want to that system so it can understand and fulfill the request.

use :after

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    .container {
      border: 1px dotted blue;
      background-color: aliceblue;
    }
    .container:after {
      content: " ";
      display: block;
      clear: both;
    }
    .float {
      width: 40%;
      float: left;
    }
  </style>
</head>
```

What's an API?

An API is a set of definitions and protocols for building and integrating application software. It's sometimes referred to as a contract between an information provider and an information user—establishing the content required from the consumer (the call) and the content required by the producer (the response). For example, the API design for a weather service could specify that the user supply a zip code and that the producer reply with a 2-part answer, the first being the high temperature, and the second being the low.

In other words, if you want to interact with a computer or system to retrieve information or perform a function, an API helps you communicate what you want to that system so it can understand and fulfill the request.

- "clearfix" technique:
 - Use the `:after` pseudo-element to insert a character space after container
 - Set its display to `"block"`
 - Clear it on both sides

Positioning Basics

- [CSS](#) provides several methods for positioning elements on the page.
- They can be positioned relative to where they would normally appear in the flow [\[relative\]](#).
- Or they can be removed from the flow altogether and placed at a particular spot on the page [\[absolute\]](#).
- You can also position an element relative to the viewport, and it will stay put while the rest of the page scrolls [\[fixed\]](#).

position

position

Values: static | relative | absolute | fixed | inherit

Default: static

Applies to: all elements

Inherits: no

top, right, bottom, left

Values: length measurement | percentage | auto | inherit

Default: auto

Applies to: positioned elements (where position value is relative, absolute, or fixed)

Inherits: no

Position Values

- **static** **Default value**. Elements render in order as they appear the document flow (normal flow).
- **relative** The element is positioned relative to its normal position (in normal flow).
- **absolute** The box is positioned exactly using x and y coordinates from the top-left corner of the containing element.
- **fixed** The position is calculated from the top-left corner of a browser window (viewport) and does not change position if the user scrolls the windows. Fixed elements are removed from the document flow and positioned relative to the viewport rather than another element in the document.
- **sticky** is a combination of relative and fixed in that it behaves as through it is relatively positioned, until it is scrolled into a specified position relative to the viewport, at which point it remains fixed.

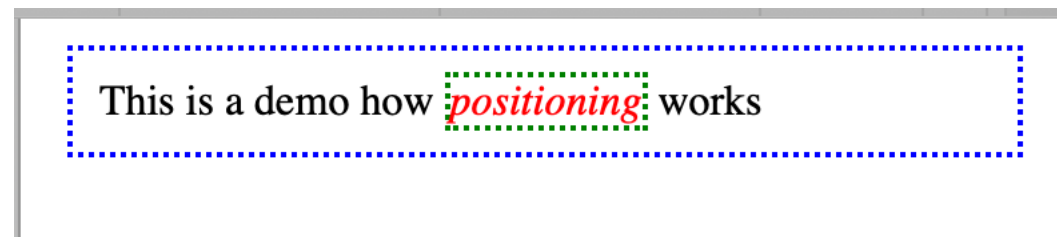
<https://medium.com/@jacobgreenaway12/taming-the-css-beast-master-positioning-5882bad14458>

position

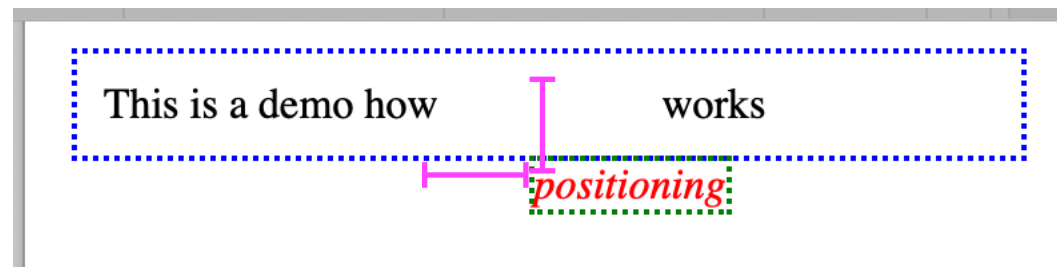
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style type="text/css">
      p {
        border: 2px dotted blue ;
        margin: 10px ;
        padding: 10px ;
      }
      em {
        border: 2px dotted green ;
        color: red ;
        position: static ;
        left: 30px;
        top: 30px;
      }
    </style>
  </head>
  <body>
    <p>This is a demo how <em>positioning</em> works</p>
  </body>
</html>
```

position

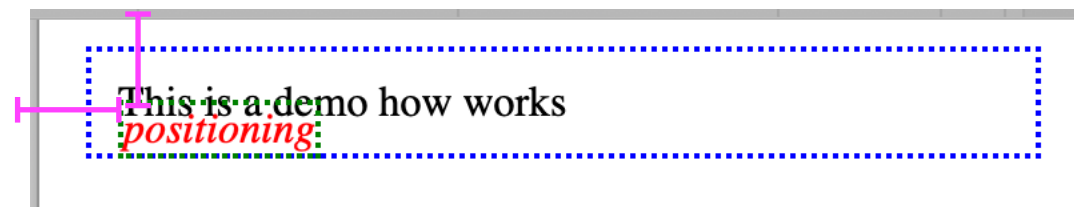
static



relative



absolute



Demo

position: fixed

```
<style>
  .button {
    border-radius: 50% ;
    text-decoration: none ;
    font-size: 40px ;
    width: 30px ;
    height: 30px ;
    text-align: center ;
    border: 2px solid blue ;
    padding: 0px 10px 20px 10px ;
    position: fixed ;
    right: 50px;
    bottom: 50px;
    background-color: lightblue ;
  }
</style>
</head>
<body>
  <a class="button" href="#" >+</a>
```

The White Rabbit Alice, a young girl, sits bored by a riverbank and spots a White Rabbit with a pocket watch and waistcoat lamenting that he is late. Surprised, Alice follows him down a rabbit hole, which sends her into a lengthy plummet but to a safe landing. Inside a room with a table, she finds a key to a tiny door, beyond which is a garden. While pondering how to fit through the door, she discovers a bottle labelled "Drink me". Alice drinks some of the bottle's contents, and to her astonishment, she shrinks small enough to enter the door. However, she had left the key upon the table and cannot reach it. Alice then discovers and eats a cake labelled "Eat me", which causes her to grow to a tremendous size. Unhappy, Alice bursts into tears, and the passing White Rabbit flees in a panic, dropping a fan and two gloves. Alice uses the fan for herself, which causes her to shrink once more and leaves her swimming in a pool of her own tears. Within the pool, Alice meets various animals and birds, who convene on a bank and engage in a "Caucus Race" to dry themselves. Following the end of the race, Alice inadvertently frightens the animals away by discussing her cat.

The Cheshire Cat The White Rabbit appears looking for the gloves and fan. Mistaking Alice for his maidservant, he orders her to go to his house and retrieve them. Alice finds another bottle and drinks from it, which causes her to grow to such an extent that she gets stuck in the house. Attempting to extract her, the White Rabbit and his neighbours eventually take to hurling pebbles that turn into small cakes. Alice eats one and shrinks herself, allowing her to flee into the forest. She meets a Caterpillar seated on a mushroom and smoking a hookah. During the Caterpillar's questioning, Alice begins to admit to her current identity crisis, compounded by her inability to remember a poem. Before crawling away, the Caterpillar says that a bite of one side of the mushroom will make her larger, while a bite from the other side will make her smaller. During a period of trial and error, Alice's neck extends between the treetops, frightening a pigeon who mistakes her for a serpent. After shrinking to an appropriate height, Alice arrives at the home of a Duchess, who owns a perpetually grinning Cheshire Cat. The Duchess's baby, whom she hands to Alice, transforms into a piglet, which Alice releases into the woods. The Cheshire Cat appears to Alice and directs her toward the Hatter and March Hare before disappearing, leaving his grin behind. Alice finds the Hatter, March Hare, and a sleepy Dormouse in the midst of a tea party. The Hatter explains that it is always 6 p.m. (tea time), claiming that time is standing still as punishment for the Hatter trying to "kill it". A conversation ensues around the table, and the riddle "Why is a raven like a writing desk?" is brought up. Alice impatiently decides to leave, calling the party stupid.

Alice trying to play croquet with a Flamingo Noticing a door on a tree, Alice passes through and finds herself back in the room from the beginning of her journey. She takes the key and uses it to open the door to the garden, which turns out to be the croquet court of the Queen of Hearts, whose guard consists of living playing cards. Alice participates in a croquet game, in which hedgehogs are used as balls, flamingos are used as mallets, and soldiers act as hoops. The Queen is short-tempered and constantly orders beheadings. When the Cheshire Cat appears as only a head, the Queen orders his beheading, only to be told that such an act is impossible. Because the cat belongs to the King, it cannot be executed.

Demo

Stacking Order

- Absolutely positioned elements overlap other elements
 - so it follows that multiple positioned elements that have the potential to stack up on one another.
- By default, elements stack up in the order in which they appear in the document.
- To change the stacking order, use **z-index** property.

z-index

z-index

Values: number | auto

Default: auto

Applies to: positioned elements

Inherits: no

- The value of the z-index is a number (positive or negative).
- The higher the number, the higher the element will appear in the stack (that is closer to you).
- Lower numbers and negative values move the element lower in the stack.

z-index property

THE MARKUP

```
<p id="A"></p>
<p id="B"></p>
<p id="C"></p>
```

THE STYLES

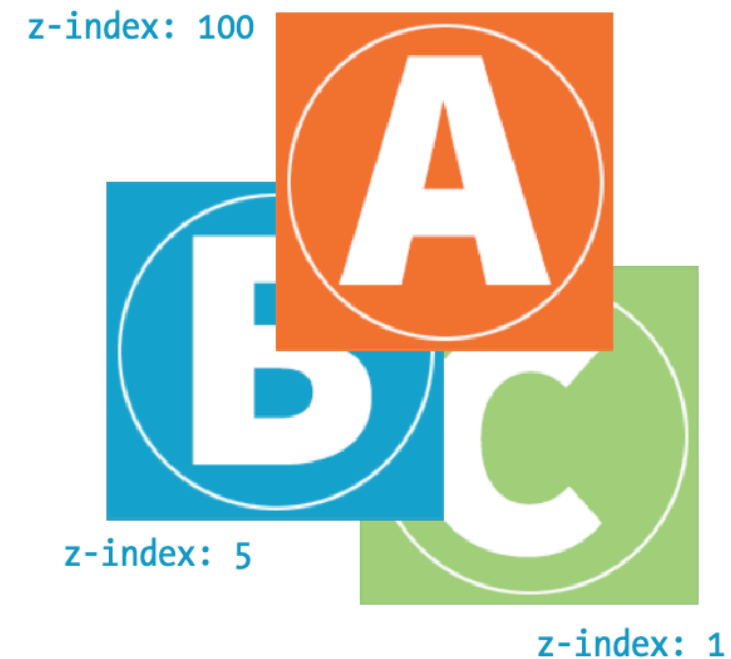
```
#A {
  z-index: 100;
  position: absolute;
  top: 175px;
  left: 200px;
}
```

```
#B {
  z-index: 5;
  position: absolute;
  top: 275px;
  left: 100px;
}
```

```
#C {
  z-index: 1;
  position: absolute;
  top: 325px;
  left: 250px;
}
```



By default, elements later in the document source order stack on top of preceding elements.



You can change the stacking order with the z-index property. Higher values stack on top of lower values.

FIGURE 15-26. Changing the stacking order with the **z-index** property.